Comp 4300 Assignment 1 HTTP Proxy Write Up
Keenan Gowdar – 7861237

SimpleHTTPProxy is a C++ HTTP GET intermediary. It accepts a client TCP connection, validates the request, establishes a separate TCP connection to the Host destination, forwards the GET, inspects the response for "RFC " followed by three digits, replaces each match with the rightmost seven student-ID digits, and returns the modified response to the client.

Compilation and Execution

Compile the three source files and start the proxy in one terminal. In a second terminal, use curl as the HTTP client. Steps to execute:

PROXY: $ clang++ main.cpp HTTPProxyServer.cpp HTTPRequest.cpp -o main
PROXY: $ ./main
CLIENT: $ curl -v -x http://localhost:11237 http://www.faqs.org/rfcs/rfc768.html

The listening port is 10000 + 1237 = 11237 (last four student-ID digits). The sample client connects over 127.0.0.1; Ctrl+C stops the proxy.

Major Components and Error Handling

Main	Serves as the main entry point into the Proxy. Declares what the studentID is, creates the HTTPProxyServer object, and sets it in motion by calling beginListening()
HTTPRequest	Parses and validates incoming requests; rejects unsupported HTTP methods.
HTTProxyServer	Accepts client TCP connections, opens an "upstream" connection, forwards responses, and closes sockets

Core Methods	handleRequest() forwards GET;
	receiveResponse() gathers the reply;
	modifyResponse() replaces matches;
	sendResponse() returns the result

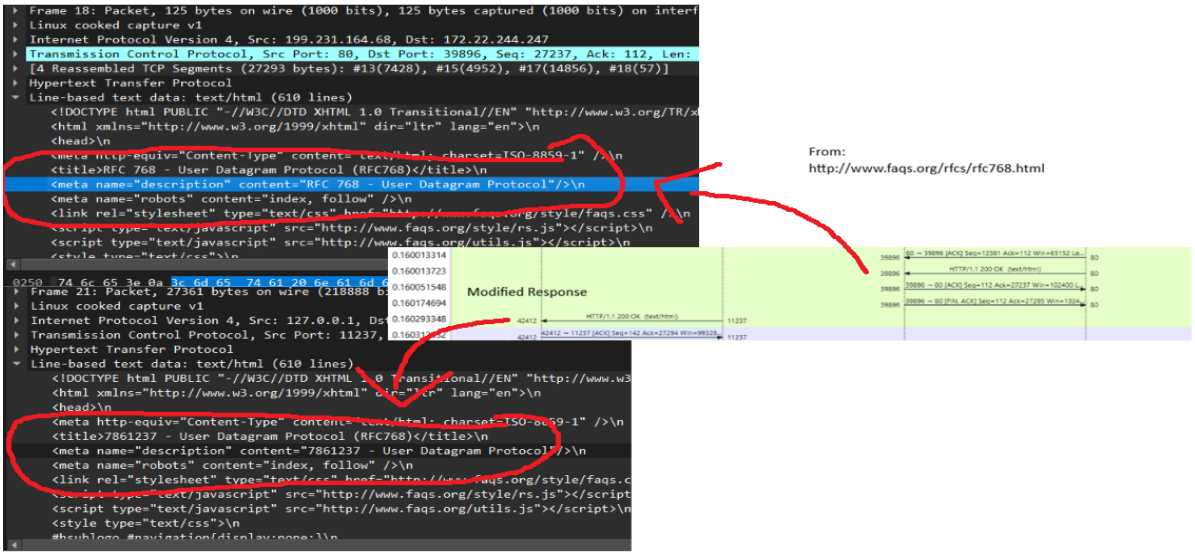
Communication Path and Evidence of Listening Port 11237 from client → Proxy → Server and back

Using the following curl command, the diagram showcases the network traffic.$curl -v -x http://localhost:11237 http://www.faqs.org/rfcs/rfc768.html



(1) curl opens TCP to 127.0.0.1:11237; (2) the proxy opens a separate connection to destination port 80 and forwards GET; (3) HTTP 200 returns to curl; (4) the final upstream ACK acknowledges the server FIN. Two distinct TCP connections are visible

Evidence of modifications of response

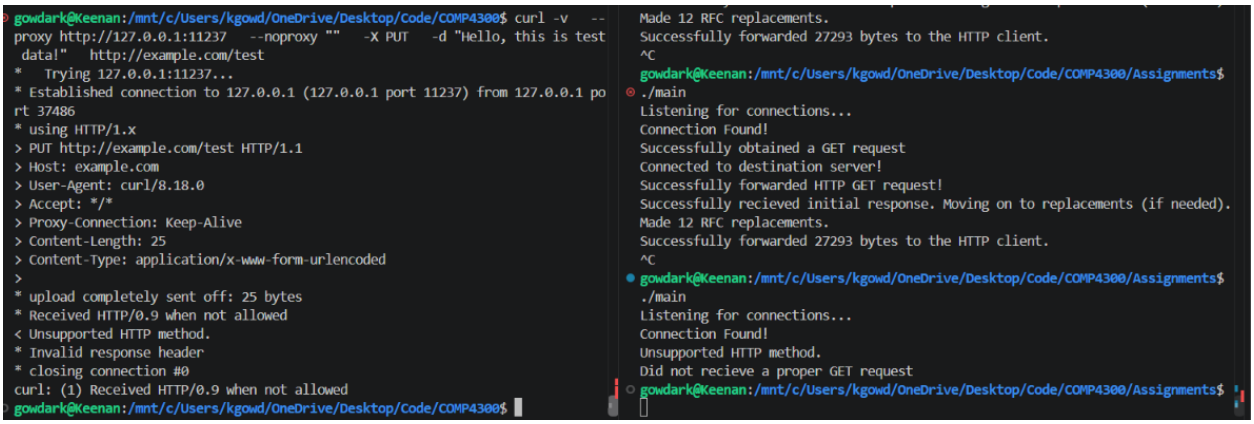


Top: the origin server sends HTML containing “RFC 768 - User Datagram Protocol.” Bottom:

the proxy sends “7861237 - User Datagram Protocol” to the client. “RFC768” (without a space) is unchanged. The proxy terminal reports 12 replacements for this request.

Evidence of Error Handling

The following PUT request was sent to the proxy (using the curl command below), with the following reply:
$curl -v --proxy http://127.0.0.1:11237 --noproxy "" -X PUT -d "Hello, this is test data!" http://example.com/test



Left: curl displays “Unsupported HTTP method.” Right: the proxy logs the rejection. The returned text is not a complete HTTP status response; curl therefore reports “Received HTTP/0.9 when not allowed.”

Refer to HTTPRequest::parse() in HTTPRequest.cpp for implementation.